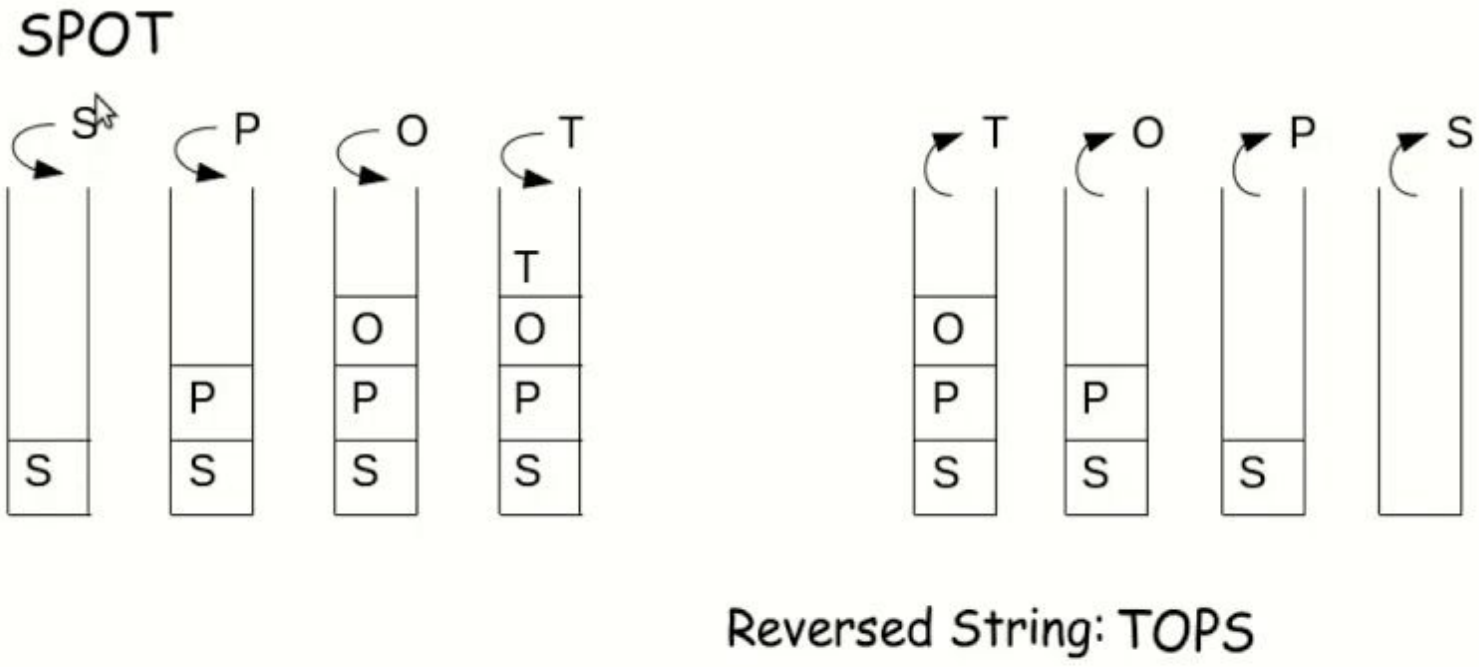
# Data Structures

## Lecture 9
## Stack Application (Continued)

Prantik Paul [PNP]
Lecturer
Department of Computer Science and Engineering
BRAC University

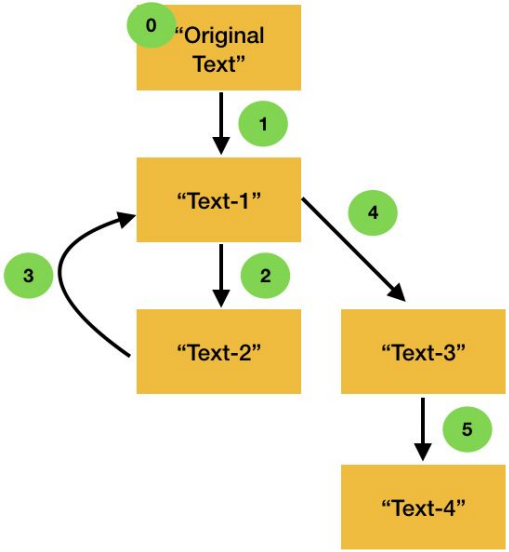
# Stack Applications (Reverse String)

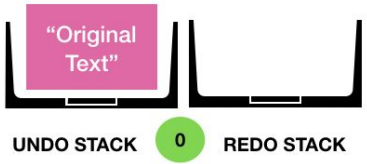


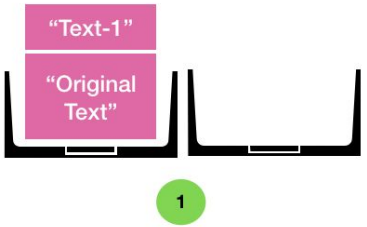
# Stack Applications (Reverse String)

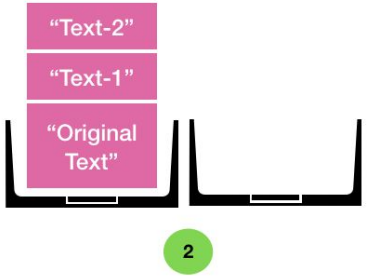
```
begin with an empty stack and an input stream.
while there is more characters to read, do:
        read the next input character;
        push it onto the stack;
end while;
while the stack is not empty, do:
        c = pop the stack;
        print c;
end while;
```

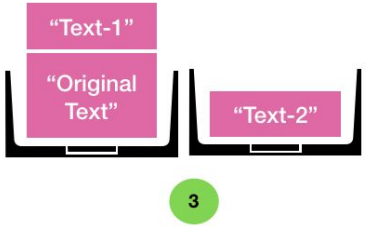
# Stack Applications (Undo Operation)

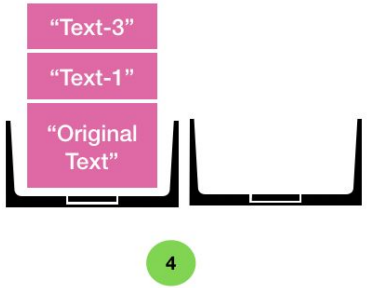


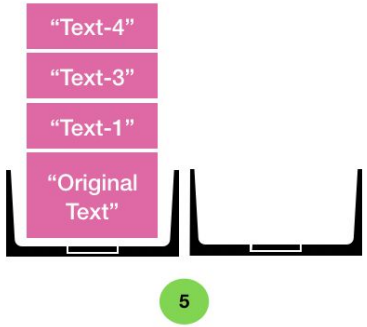



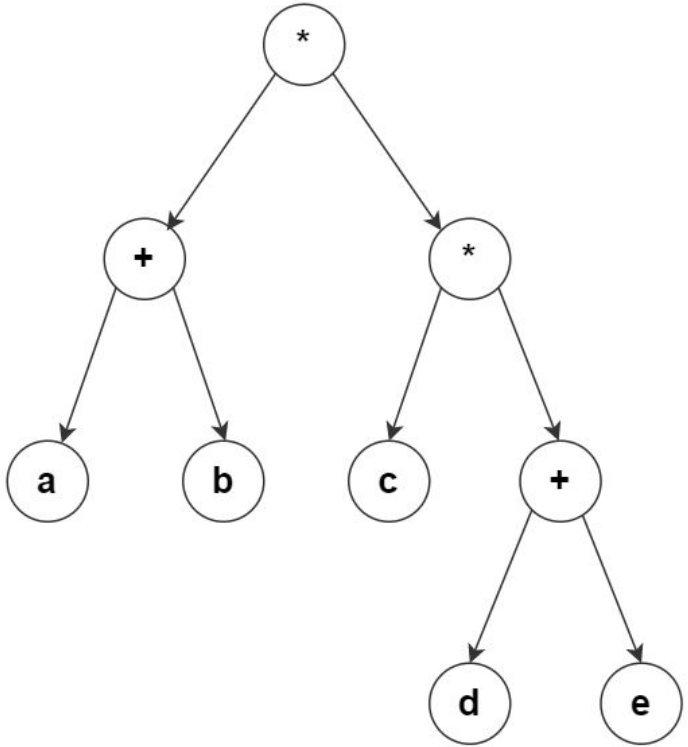



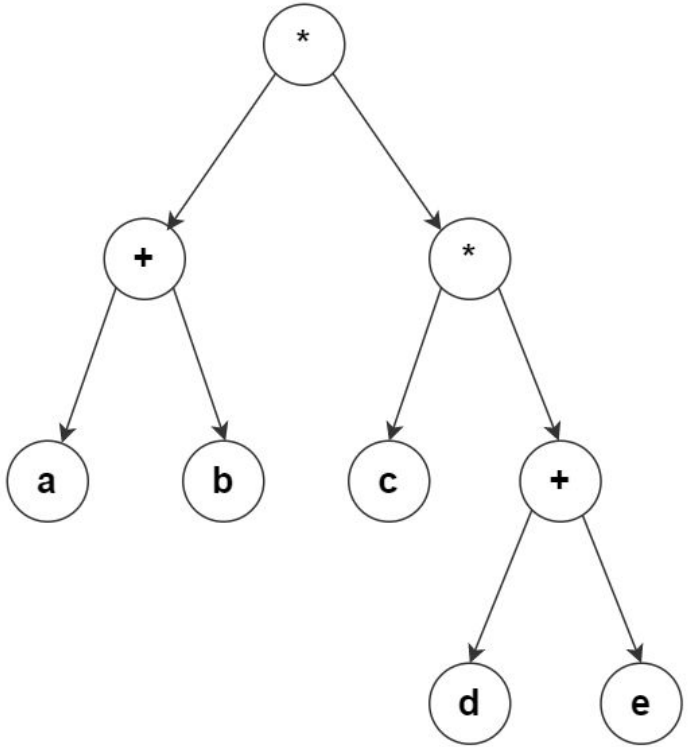



# Stack Applications (Expression Tree)




(a+b) * (c* (d+e) )

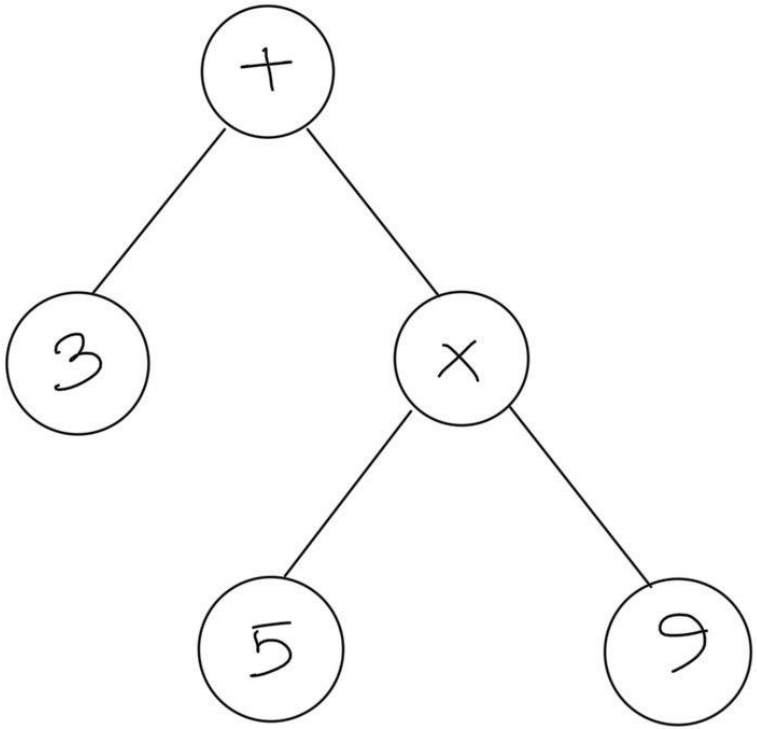
# Stack Applications (Postfix Notation)




(a+b)*(c*(d+e))

a b + c d e + * *

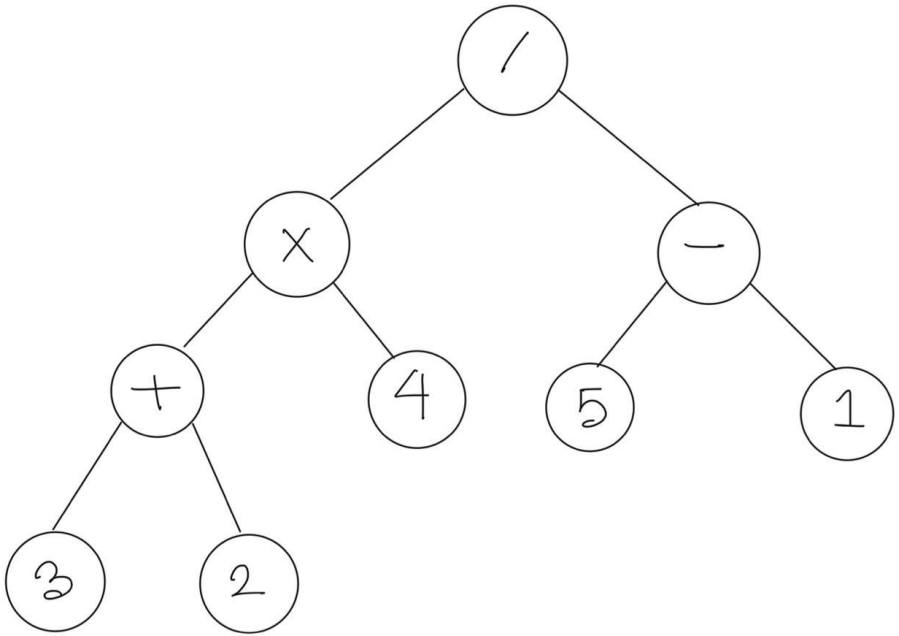
# Stack Applications (Postfix Notation)




3 + 5 * 9

3 5 9 * +

# Stack Applications (Postfix Notation)




```
((3 + 2) * 4) / (5 - 1)
```

```
3 2 + 4 * 5 1 - /
```

# Stack Applications (Expression Evaluation)

```
 Stack    Expression

|   |
 ---        3 2 + 4 * 5 1 - /
(empty)

| 3 |       2 + 4 * 5 1 - /
 ---

| 2 |
| 3 |       + 4 * 5 1 - /
 ---

| 5 |       4 * 5 1 - /
 ---

| 4 |
| 5 |       * 5 1 - /
 ---

| 20|       5 1 - /
 ---

| 5 |
| 20|       1 - /
 ---
```

```
| 1 |
| 5 |
| 20|         - /
 ---

| 4 |
| 20|         /
 ---

| 5 |        (finished.
 ---
```

# Stack Applications (Expression Evaluation)

```
begin with an empty stack and an input stream (for the expression).
while there is more input to read, do:
        read the next input symbol;
        if it's an operand,
            then push it onto the stack;
        if it's an operator
            then pop two operands from the stack;
            perform the operation on the operands;
            push the result;
end while;
// the answer of the expression is waiting for you in the
stack: pop the answer;
```